

A

DEVELOPER GUIDE

AisleFinder Full-Stack Development Guide

A complete walkthrough of the app's stack — React frontend, Flask API, the Kroger integration, AWS accounts, and shipping to web, iOS, and Android — using this codebase as the running example.

13 chapters

Backend to app store

Kroger API

Product search & store lookup

React + Flask

Capacitor for iOS / Android

CONTENTS

Everything in this guide

THE BACKEND

[01 Python Server](#)

[02 The Kroger API](#)

[03 Testing the Python Backend](#)

THE FRONTEND

[04 React Frontend](#)

[05 Testing the React Frontend](#)

[06 The Design System](#)

SHIPPING IT

[07 Web Deployment](#)

[08 Adding iOS Support](#)

[09 Adding Android Support](#)

GOING MULTI-USER

[10 Adding AWS Logins](#)

[11 Offline-First Sync](#)

[12 The Full Codebase](#)

RUNNING IT SAFELY

[13 Security & App Hardening](#)

01 Python Server

The backend of Aisle Finder is a **Flask** server written in Python. Its job: take a raw grocery list, look up each item against Kroger's product API, and return the list reorganized by aisle. When accounts are configured (chapter 10), it also stores and shares lists.

1.1 Basic Python setup

Python is an interpreted language — you don't compile it, you just run `.py` files with an interpreter. macOS ships with a system Python, but you should never install packages into it; use your own install (Homebrew is easiest):

```
# Run this
brew install python3
python3 --version # anything 3.10+ is fine
```

Two commands to know:

- `python3 some_file.py` — run a script
- `pip3 install some_package` — install a library (but see `venvs` below first!)

Python finds importable code via `sys.path`. Aisle Finder's server adds the repo root to that path at startup so `grocery_organizer/...` imports work (`api_server.py:20–21`):

```
project_root = Path(__file__).parent
sys.path.append(str(project_root))
```

1.2 Virtual environments

If you `pip install` globally, every project on your machine shares one pile of packages — project A needs `Flask 3.0`, project B needs `Flask 2.x`, and one of them breaks. A **virtual environment (venv)** is a private, per-project copy of Python + packages that lives in a folder inside the repo.

```
# Run this – from the repo root
python3 -m venv venv # create the venv (a ./venv folder appears)
source venv/bin/activate # activate it for this terminal session
```

While a `venv` is active your prompt shows `(venv)`, and `python` / `pip` point at the private copy. Install Aisle Finder's dependencies into it:

```
# Run this – with (venv) active
pip install -r requirements.txt
```

`requirements.txt` is the standard "shopping list" of dependencies:

```
Flask==3.0.0           # the web framework
Flask-CORS==4.0.0      # lets the React dev server (a different port) call the API
requests==2.31.0       # HTTP client used to call the Kroger API
python-dotenv==1.0.0   # loads .env files into environment variables
boto3>=1.34.0          # AWS SDK (DynamoDB + Cognito, chapter 10)
```

To leave the venv later: `deactivate` . To come back: `source venv/bin/activate` . The `venv/` folder is git-ignored — every developer creates their own.

1.3 Secrets and the `.env` file

The server reads credentials from **environment variables**, loaded from a `.env` file at the repo root by `python-dotenv` (`api_server.py:17`):

```
from dotenv import load_dotenv
load_dotenv()
```

The one variable required for the core feature is the Kroger API secret (`grocery_organizer/src/store_api/kroger.py`):

```
client_secret = os.getenv('KROGER_CLIENT_SECRET')
if not client_secret:
    raise ValueError("KROGER_CLIENT_SECRET environment variable is required")
```

Create `.env` in the repo root — `cp .env.example .env` gives you the template. It's git-ignored; never commit it:

```
KROGER_CLIENT_ID=your_client_id_here
KROGER_CLIENT_SECRET=your_secret_here
```

Both values come from your Kroger developer registration — Kroger's API terms don't allow committing either one, so they live only in `.env` . Chapter 2 covers where both come from.

1.4 Starting the API server

```
# Run this – venv active, .env in place
python api_server.py
```

You should see Flask start on port 8000. Verify it's alive:

```
# Run this – in a second terminal
curl http://localhost:8000/health
# → {"status": "healthy"}
```

Then try the real thing — organize a list:

```
# Run this
printf "milk\neggs\nbananas\nbread" > /tmp/list.txt
curl -X POST http://localhost:8000/api/process-grocery-list \
  -F "file=@/tmp/list.txt" \
  -F "output_format=aisle"
```

You'll get back markdown grouped by aisle (`## AISLE 3\n- bread\n...`).

1.5 How the server is organized

`api_server.py` itself is tiny — the route logic lives in shared **blueprints** (Flask's way of splitting routes across files) so the local server and the Vercel serverless copy (`api/index.py` , chapter 7) can't drift apart:

```
from grocery_routes import grocery_bp      # product lookup endpoints
from lists_backend import lists_bp        # account-backed list sync (ch. 10)

app = Flask(__name__)
CORS(app, origins=['http://localhost:3000', 'https://aislefinder3000.com',
                  'capacitor://localhost', 'http://localhost', 'https://localhost'])
app.register_blueprint(grocery_bp)
app.register_blueprint(lists_bp)
```

Note the CORS list: browsers block cross-origin requests by default, and in local dev React runs on port 3000 while Flask runs on 8000 — different origins. `Flask-CORS` explicitly allowlists the React dev server, the production domain, and the Capacitor app origins (chapters 8–9).

The endpoints in `grocery_routes.py` :

ROUTE	METHOD	PURPOSE
<code>/api/process-grocery-list</code>	POST	Upload a list file → aisle-organized markdown
<code>/api/find-stores</code>	POST	ZIP code → nearby Kroger stores
<code>/api/find-item-aisle</code>	POST	One item → its aisle at a given store
<code>/api/health</code>	GET	Health check

A request flows: route → `GroceryListProcessor` (`grocery_organizer/src/core/processor.py`) → `KrogerAPI` product lookups in parallel → `OutputFormatter` renders markdown. The Kroger client is the interesting part — that's chapter 2.

TODOs to get this working

- ☐ **Install Python 3.10+** — `brew install python3`
- ☐ **Create + activate a venv** — `python3 -m venv venv && source venv/bin/activate`
- ☐ **Install deps** — `pip install -r requirements.txt`
- ☐ **Register a Kroger developer account** at <https://developer.kroger.com>, create an app with the *Products* and *Locations* APIs, and copy its client secret (details in chapter 2)
- ☐ **Create `.env`** in the repo root with `KROGER_CLIENT_SECRET=...`
- ☐ **Start the server** — `python api_server.py` , then `curl localhost:8000/health`

02 The Kroger API

The heart of the backend: turning "2 lbs of chicken" into "Aisle 14". This chapter walks the Kroger client in `grocery_organizer/src/store_api/kroger.py` — OAuth, search cleanup, and the match-scoring logic that picks the right product.

2.1 Where it sits

`KrogerAPI` is a plain Python class constructed with a store ID and used directly by both callers:

```
# grocery_routes.py - single-item lookup
product = KrogerAPI(store_id).find_product(item)

# core/processor.py - whole-list processing
api_client = KrogerAPI(self.store_id)
api_client.get_auth_token() # pre-fetch token before parallel calls
```

It exposes two public operations: `find_product(name)` → a `FullProduct` (input name, matched description, category, aisle number), and `find_stores_by_zip(zip)` → nearby stores for the store picker.

(Historical note: the codebase once had a second store client behind a registry/interface abstraction. It was removed in July 2026 — one chain didn't justify the indirection. The git history has the pattern if a second chain ever comes back.)

2.2 Credentials and OAuth

Kroger's API uses **OAuth 2.0 client credentials** — the flow for server-to-server access where no human logs in. You register an app at <https://developer.kroger.com> and get a **client ID** and a **client secret**, both read from `.env` (`KROGER_CLIENT_ID` / `KROGER_CLIENT_SECRET`) — Kroger's API terms treat both as credentials that must stay out of the repo.

To get an access token, you POST both — joined and base64-encoded as an HTTP Basic auth header — to the token endpoint:

```
auth_code = client_id + ':' + client_secret
headers = {
    'Content-Type': 'application/x-www-form-urlencoded',
    'Authorization': 'Basic ' + base64.b64encode(auth_code.encode('utf-8')).decode('utf-8')
}
data = {'grant_type': 'client_credentials', 'scope': 'product.compact'}
response = requests.post(self.AUTH_URL, headers=headers, data=data)
```

Tokens expire, so `get_auth_token()` caches one and refreshes 60 seconds early (`self.token_expiration = time.time() + expires_in - 60`). And because item lookups run in parallel threads (chapter 1), the refresh is guarded by a `threading.Lock` with a double-check — first thread in refreshes, the rest reuse its token instead of stampeding the auth endpoint. (That's also why the processor pre-fetches the token once before fanning out.)

2.3 Cleaning the search term: `search_terms.py`

Shopping lists say "2 lbs of chicken"; search APIs match best on "chicken". `preprocess_search_term()` strips quantities, units, and filler words with regex patterns before anything is sent to Kroger:

```
cleaned = product_name.lower().strip()
for pattern in QUANTITY_PATTERNS:      # "2 lbs", "three gallons", "a dozen"...
    cleaned = re.sub(pattern, ' ', cleaned)

words = [w for w in cleaned.split() if w not in FILLER_WORDS and len(w) > 1]
result = ' '.join(words).strip()
return result if result else product_name.strip()  # never return nothing
```

Note the last line: if cleaning would delete the whole term (the item was literally "a bag"), fall back to the raw input — a worse search beats an empty one. Defensive fallbacks like this are a theme in this layer.

2.4 Picking the right match: relevance + scoring

Kroger's search often puts the wrong product first ("ice" returns rice; "lemon" returns lemon-scented candles).

`find_product()` takes the top 5 candidates and runs each through two functions:

`_is_product_relevant()` is a yes/no gate: at least half of the meaningful search words must appear in the product description, using word-level fuzzy matching (`_fuzzy_word_match`) that allows prefixes ("cheezit" ↔ "cheezits") and suffixes ("berries" in "strawberries") but rejects substring traps like "ice" inside "rice". It also rejects non-grocery products — a `NON_GROCERY_KEYWORDS` list (candles, detergent, pet food, gift cards...) blocks a match *unless the user actually searched for that keyword*.

`_score_product()` ranks the survivors. Reading the bonuses tells you what "the right product" means here:

```
score += 15  # description words == search words exactly
score += 10  # exact phrase appears in description
score += 3   # per matched search word
score += 5   # has aisleLocations (a real shelf item, not online-only)
score += 5   # fresh category (produce/bakery/deli/meat) when user didn't say "frozen"
score -= 5   # frozen category when user didn't ask for frozen
score -= 20  # non-grocery keyword in description
score += 4   # short description (< 20 chars ≈ the base product, not a variant)
```

The winner becomes a `FullProduct`; the aisle number comes from the product's `aisleLocations` for the specific `store_id`, and `-1` means unknown. If nothing passes the gate, the item becomes a "Not Found" entry rather than an error — one unmatched item never breaks the list.

This is the file to tune when matching misbehaves: add a category alias to `CATEGORY_MAP`, a blocked keyword to `NON_GROCERY_KEYWORDS`, or adjust a bonus. The 64 scoring tests in chapter 3 make that safe to do.

2.5 Resilience: retries with exponential backoff

Every network-touching method wears the `@retry_api_call` decorator:

```
@retry_api_call(max_retries=3, backoff_factor=1)
def find_product(self, product_name):
    ...
```


It retries **only** `requests.RequestException` (network flakiness), waiting 1s → 2s → 4s between attempts.

Deterministic errors — bad config, malformed responses — propagate immediately, since they'd fail identically every time. That distinction (retry the transient, fail fast on the permanent) is worth stealing for any API client you write.

2.6 Debug tooling

When a lookup returns something weird, you want to see Kroger's *raw* response. The local server (never Vercel) has two tools for this:

```
# Run this – raw top-5 results for a term at a store
curl -X POST http://localhost:8000/api/debug-kroger \
  -H "Content-Type: application/json" \
  -d '{"term": "cheez its", "store_id": "01400943"}'
```

Or open <http://localhost:8000/debug> — a small HTML viewer (`api-response-viewer.html`) around the same endpoint.

TODOs to get this working

- ☐ **Create a Kroger developer account** at <https://developer.kroger.com>
- ☐ **Register an application** with access to the *Products* and *Locations* APIs (instant approval for these public APIs)
- ☐ **Put your client ID and secret** in `.env` as `KROGER_CLIENT_ID=...` and `KROGER_CLIENT_SECRET=...` (and later in Vercel, chapter 7)
- ☐ **Verify end to end** — start the server and hit `/api/find-item-aisle` with `{"item": "milk", "store_id": "01400943"}`, or poke the raw API at <http://localhost:8000/debug>

03 Testing the Python Backend

The backend has a **pytest** suite in `grocery_organizer/tests/` — about a hundred tests that run in seconds with zero network access. This chapter covers running them, and the two testing techniques the suite is built on.

3.1 Running the suite

pytest is a dev-only tool, so it's not in `requirements.txt`; install it into your venv once:

```
# Run this – venv active
pip install pytest
```

```
# Run this – from the repo root
PYTHONPATH=. python -m pytest grocery_organizer/tests
```

`PYTHONPATH=.` matters: the code imports as `grocery_organizer.src...`, so the repo root must be on Python's import path (the server does this itself at startup; for pytest you supply it). Useful variations:

```
# Run this – one file, verbose
PYTHONPATH=. python -m pytest grocery_organizer/tests/test_search_terms.py -v

# Run this – only tests whose name matches a keyword
PYTHONPATH=. python -m pytest grocery_organizer/tests -k "frozen"
```

pytest's model is simple: any `test_*.py` file, any `test_*` function, and plain `assert` statements — no assertion classes to memorize.

3.2 What the suite covers

FILE	TESTS	COVERS
<code>test_word_matching.py</code>	64	Kroger fuzzy matching + scoring (chapter 2's <code>_fuzzy_word_match</code> , <code>_is_product_relevant</code> , <code>_score_product</code>)
<code>test_input_parser.py</code>	13	Text → items: newlines, commas, bullets, checkboxes, numbering
<code>test_grocery_routes.py</code>	9	The Flask endpoints, with the Kroger client stubbed
<code>test_output_formatter.py</code>	7	Grouping, food-safe section order, markdown rendering
<code>test_search_terms.py</code>	3	Quantity/filler stripping

Notice where the weight is: 64 of 96 tests target the match-scoring logic — the code with the most edge cases and the most reasons to change (every tuning tweak from chapter 2 risks regressing an earlier fix). Test density should follow fragility, not file size.

3.3 Technique 1: parametrize — one test, many cases

Most of the suite is input/output pairs, and pytest has first-class support for that shape (`test_search_terms.py`):

```
@pytest.mark.parametrize("raw,expected", [
    ("2 lbs chicken", "chicken"),
    ("3 gallons of milk", "milk"),
    ("a dozen eggs", "eggs"),
    ("two loaves of bread", "bread"),
    ("large eggs", "eggs"),
    ("peanut butter", "peanut butter"),
    ("Cheez-Its", "cheez-its"),
])
def test_quantities_and_fillers_removed(raw, expected):
    assert preprocess_search_term(raw) == expected
```

Each tuple runs as a separate test with its own pass/fail. When you find a mis-parsed item in real use, the fix is one new tuple — this is how the word-matching file grew to 64 cases.

3.4 Technique 2: stubbing — testing routes without Kroger

Route tests must not call the real Kroger API (slow, flaky, needs secrets). So the tests swap the whole client out for a fake (`test_grocery_routes.py`):

```
class StubKrogerAPI:
    """Deterministic stand-in for KrogerAPI."""

    def get_auth_token(self):
        return 'stub-token'

    def find_product(self, product_name):
        if product_name == 'milk':
            return FullProduct('milk', 'Whole Milk', 'Dairy', -1)
        ...

@pytest.fixture
def client():
    app = Flask(__name__)
    app.register_blueprint(grocery_bp)
    # Both the routes and the processor import KrogerAPI by name
    with patch.object(grocery_routes, 'KrogerAPI', StubKrogerAPI), \
        patch.object(processor, 'KrogerAPI', StubKrogerAPI):
        with app.test_client() as test_client:
            yield test_client
```

Three ideas packed in here:

- A **stub** is a hand-written fake with the same methods as `KrogerAPI` but canned answers — the whole test becomes deterministic.
- `patch.object` swaps the `KrogerAPI` name for the stub *for the duration of the test only*. Both modules that imported it get patched — a classic gotcha: patch where a name is *used*, not where it's defined.
- A **fixture** is pytest's reusable setup: any test that takes a `client` argument receives this pre-stubbed Flask **test client**, which fakes HTTP requests in-process — no server, no port:

```
def test_returns_organized_markdown(self, client):
    response = client.post('/api/process-grocery-list', data={
        'file': (io.BytesIO(b'milk\nrice'), 'list.txt'),
        'output_format': 'aisle',
    })
    assert response.status_code == 200
```

This layering works because `grocery_organizer/` never imports Flask, and all network access is funneled through one class — each layer is testable at its own boundary.

3.5 Writing a new test

The habits this codebase follows:

1. **Pure logic** (parser, formatter, scoring) → plain function tests, parametrized when there are multiple cases.
2. **Route behavior** → add to `test_grocery_routes.py`; the `client` fixture already isolates you from the network.
3. **Fixing a bug?** Write the failing case first, watch it fail, then fix — the test is now permanent protection against regression.
4. Name tests as sentences: `test_falls_back_to_original_when_everything_filtered` reads as documentation when it fails.

TODOs to get this working

- ☐ **Install pytest** — `pip install pytest` (venv active)
- ☐ **Run the suite** — `PYTHONPATH=. python -m pytest grocery_organizer/tests` and confirm everything passes (no `.env` or network needed)
- ☐ **Try a mutation** — flip a score bonus in `api.py`, rerun, and watch `test_word_matching.py` catch it; revert
- ☐ **Adopt the rhythm** — run the suite before every commit that touches `grocery_organizer/` or the blueprints

04 React Frontend

The user interface is a **React** single-page app. React's core idea: the UI is a function of state. You write **components** (JavaScript functions that return HTML-like markup called JSX), React re-renders them whenever their state changes, and it efficiently updates only the parts of the page that changed.

4.1 The framework: Create React App

Aisle Finder uses **Create React App (CRA)** — a zero-config toolchain that bundles React, a dev server with hot reload, and a production build step. Everything is driven by npm scripts in `package.json`:

```
"scripts": {
  "start": "react-scripts start",    // dev server on http://localhost:3000
  "build": "react-scripts build",    // production bundle → build/
  "test": "react-scripts test"      // run the test suite (chapter 5)
}
```

```
# Run this – from the repo root
npm install    # install dependencies into node_modules/ (one-time)
npm start     # opens http://localhost:3000 with hot reload
```

To talk to your local Flask server from chapter 1, point the frontend at it via a `REACT_APP_*` environment variable in `.env`:

```
REACT_APP_API_URL=http://localhost:8000
```

CRA bakes `REACT_APP_*` vars in **at build/start time** — restart `npm start` after editing `.env`. `src/api.js` reads it to build request URLs and attaches auth headers when needed:

```
const jsonHeaders = (token) => ({
  'Content-Type': 'application/json',
  ...(token ? { Authorization: `Bearer ${token}` } : {}),
});
```

4.2 Anatomy of a component: `Sheet.jsx`

The simplest real component in the codebase is the bottom-sheet wrapper (`src/components/Sheet.jsx`) — it demonstrates props, conditional rendering, JSX, and event handling in 40 lines:

```
// Bottom sheet used for account, share, and store pickers
const Sheet = ({ open, onClose, children }) => {
  if (!open) return null;
  return (
    <div
      onClick={(e) => { if (e.target === e.currentTarget) onClose(); }}
      style={{
        position: 'fixed',
        inset: 0,
        background: 'var(--af-backdrop)',
        display: 'flex',
        alignItems: 'flex-end',
        justifyContent: 'center',
      }}
    >
      <div style={{ /* ...the sheet panel... */ }}>
        {children}
      </div>
    </div>
  );
};
```

What to notice:

- **Props** — `{ open, onClose, children }` are inputs passed by the parent. `children` is whatever JSX the parent nests inside `<Sheet>...</Sheet>`.
- **Conditional rendering** — `if (!open) return null;` means "render nothing." No imperative show/hide; the parent just flips a boolean.
- **Data flows down, events flow up** — the sheet never decides to close itself; it calls `onClose()` (a function handed down by the parent), and the parent changes state, which re-renders with `open={false}`.
- **The backdrop-click trick** — `e.target === e.currentTarget` ensures only clicks on the backdrop itself (not on content inside it) close the sheet.

4.3 State and hooks: `useAuth` in `auth.js`

Hooks are functions starting with `use` that give components state and lifecycle behavior. `useState` holds a value across re-renders; `useEffect` runs side effects (fetching, subscriptions) after render. A **custom hook** bundles these into a reusable unit — here's the app's auth hook (`src/auth.js:90`):

```
export const useAuth = () => {
  const [user, setUser] = useState(null);
  const [authLoading, setAuthLoading] = useState(authConfigured);

  useEffect(() => {
    if (!authConfigured) return;
    let cancelled = false;
    getCurrentUser().then((u) => {
      if (!cancelled) {
        setUser(u);
        setAuthLoading(false);
      }
    });
    return () => { cancelled = true; };
  }, []);
  // ...returns { user, authLoading, signIn, signOut, signUp, confirmSignUp }
};
```

What to notice:

- `useState(null)` — `user` starts as `null` (guest mode); calling `setUser` triggers a re-render everywhere `user` is read.
- `useEffect(..., [])` — the empty dependency array means "run once when the component mounts": check whether a Cognito session already exists.
- The `cancelled` flag + cleanup function is the standard guard against setting state on a component that unmounted while the promise was pending.

The heavyweight sibling is `useLists(user)` in `src/listsStore.js` — the app's entire data layer as one hook. Chapter 11 dissects it.

4.4 Composition: the shell in `AisleFinder.jsx`

`src/AisleFinder.jsx` is the root component. It owns two pieces of navigation state — which **screen** is showing and which **sheet** (modal) is open — and composes everything else:

```
AisleFinder (shell: theme CSS, toast, online/offline, screen + sheet state)
├─ TopBar                               - logo, user name, nav buttons
├─ one of four screens:
│   ├─ CurrentListScreen                 - quick-add bar, items, frequent suggestions
│   ├─ MyListsScreen                     - switch / create / delete lists
│   ├─ HistoryScreen                     - completed trips by month
│   └─ ShopScreen                         - aisle groups, drag-drop, check-off, confetti
└─ sheets (each wrapped in <Sheet>):
    ├─ AccountSheet                      - sign in / sign up / confirm code
    ├─ ShareSheet                         - create + join share codes
    └─ StoreSheet                         - ZIP search, pick your store
```

This is React's composition model in miniature: no router library — the screen is just a state variable, and rendering is a switch over it. State that multiple screens need (`lists` , `user`) lives in the shell and is passed down as props.

The shell also owns the app's visual identity — theme CSS variables, fonts, icons. Those conventions get their own chapter (chapter 6); read it before touching any UI code.

FILE	ROLE
<code>index.js</code>	CRA entry point — mounts <code><AisleFinder /></code> into the page
<code>AisleFinder.jsx</code>	Shell: theme, navigation state, composition
<code>listsStore.js</code>	<code>useLists()</code> — the data layer + background sync (ch. 11)
<code>auth.js</code>	<code>useAuth()</code> — Cognito sign-in (chapter 10)
<code>api.js</code>	All <code>fetch</code> calls to the Flask backend
<code>storage.js</code>	<code>localStorage</code> , mirrored to native storage on mobile (ch. 8–9)
<code>listUtils.js</code>	Parse/build the backend's markdown list format
<code>screens/</code>	The four full-page views
<code>components/</code>	Reusable pieces: <code>TopBar</code> , <code>Sheet</code> , the three sheets, <code>Logo</code>

TODOs to get this working

- ☐ **Install Node.js** (18+) — `brew install node`, verify with `node --version`
- ☐ **Install dependencies** — `npm install` in the repo root
- ☐ **Point at your backend** — add `REACT_APP_API_URL=http://localhost:8000` to `.env` (the same file from chapter 1)
- ☐ **Start both servers** — `python api_server.py` in one terminal, `npm start` in another, open <http://localhost:3000>
- ☐ **Smoke test** — add "milk, eggs, bread" via the quick-add bar, pick a store in the store sheet, hit Shop, and confirm items come back grouped by aisle

05 Testing the React Frontend

The frontend has a **Jest** suite — two files, two very different styles of test, and both run without a browser.

5.1 Running the tests

Jest ships inside CRA; there's nothing to install. Two modes:

```
# Run this – interactive watch mode: reruns tests as you edit
npm test

# Run this – single pass and exit (what CI and pre-commit checks want)
CI=true npm test
```

There's no browser involved: Jest runs in Node with **jsdom**, a simulated DOM. Components really render, `document` and `localStorage` really exist — it's just headless and fast. The trade-off: nothing is *painted*, so tests assert on DOM content, not pixels.

5.2 Style 1: pure-function tests (`listUtils.test.js`)

The easiest code to test is code with no React in it — plain functions in, values out. `src/listUtils.js` (parsing the backend's `## Section\n- item` markdown into groups and back) is tested exactly like the Python `search_terms` tests in chapter 3: feed input, assert output, no setup.

This is a design lesson as much as a testing one: because the markdown logic lives in a plain module instead of inside a component, testing it costs nothing. Keep logic out of components and most of your tests stay this simple.

5.3 Style 2: rendering the whole app (`AisleFinder.test.js`)

The second file mounts the *entire* app and asserts on what appears:

```
const render = () => {
  const div = document.createElement('div');
  document.body.appendChild(div);
  act(() => { createRoot(div).render(<AisleFinder />); });
  return div;
};

beforeEach(() => {
  localStorage.clear();
  document.body.innerHTML = '';
});

it('renders the current-list home screen in guest mode', () => {
  const div = render();
  expect(div.textContent).toContain('AisleFinder');
  expect(div.textContent).toContain('Guest');
  expect(div.querySelector('input[placeholder*="Add an item"]')).not.toBeNull();
});
```

What to notice:

- `act(...)` wraps the render so React flushes all state updates and effects before your assertions run — without it, you'd assert against a half-rendered tree (React warns loudly when you forget).
- `beforeEach` **resets shared state**. The app persists to `localStorage` on every change (chapter 11), and `jsdom`'s `localStorage` survives between tests in a file. Without the reset, test A's lists leak into test B — the classic source of tests that pass alone and fail together.
- **Tests always run in guest mode** — `jsdom` starts with no saved Cognito session, so `useAuth` resolves to no user and nothing touches the network.

5.4 The migration test: pinning down risky code

The best test in the suite protects the scariest code path — upgrading a user's stored data from the old single-list format without losing anything:

```
it('migrates the old single-list localStorage format', () => {
  localStorage.setItem('af_textInput', JSON.stringify('milk\neggs, bananas'));
  localStorage.setItem('af_groceryList', JSON.stringify('## Dairy\n- milk\n- eggs\n\n## Produce\n- milk\n- eggs\n- bananas'));
  const div = render();
  expect(div.textContent).toContain('milk');
  const lists = JSON.parse(localStorage.getItem('af_lists'));
  expect(lists).toHaveLength(1);
  expect(lists[0].items.map((it) => it.name).sort()).toEqual(['bananas', 'eggs', 'milk']);
  expect(lists[0].organized).toContain('## Dairy');
});
```

The pattern: *seed* the pre-condition (old-format keys in `localStorage`), *run* the system (mount the app), *assert* the outcome at both levels — what the user sees (`textContent`) and what got persisted (`af_lists`). Note what it does **not** assert: no internal function calls, no component structure. It tests behavior, so refactoring `listsStore.js` freely won't break it — only actually losing user data will.

5.5 When you add frontend code

- New parsing/formatting logic → put it in `listUtils.js` (or a new plain module) and test it pure.

- New user-visible flows → extend `AisleFinder.test.js` : seed `localStorage`, render, assert on the DOM.
- Anything touching stored data shapes → **always** add a migration test like §5.4; `localStorage` on a user's phone is forever.

TODOs to get this working

- ☐ **Run the suite once** — `CI=true npm test` (needs only `npm install` from chapter 4; no backend, no `.env`)
- ☐ **Try watch mode** — `npm test` , then edit a file and watch it rerun
- ☐ **Adopt the rhythm** — `CI=true npm test` before every commit that touches `src/` , alongside the Python suite from chapter 3

06 The Design System

Aisle Finder has a deliberately small visual language, enforced by a handful of hard rules. This chapter explains the token system and the rules — read it before writing any UI code. The visual reference lives at `docs/design-rules.html`; keep that file in sync with any theme change.

6.1 Design tokens: the `--af-*` CSS variables

Every color in the app flows through **CSS custom properties** defined once in the shell (`src/AisleFinder.jsx`). Components never contain hex values — they say *what role* a color plays, not what it is:

```
// in any component
background: 'var(--af-surface)',
color: 'var(--af-text-muted)',
```

The core palette is nine colors per scheme:

```
:root {
  --af-bg: #ffffff;           /* page background */
  --af-surface: #e4edf6;      /* cards, panels */
  --af-border: #d9e7f2;
  --af-text: #1c2a3a;
  --af-text-muted: #5f7183;
  --af-green: #1f5fa0;        /* the brand color – primary actions (navy, despite the name) */
  --af-green-dark: #153f6e;   /* hover states, emphasis */
  --af-amber: #ffb52e;         /* the one accent – highlights, stars */
  --af-error-text: #b3541e;
}
```

Everything else is **derived** — either an alias (`--af-focus: var(--af-green)`) or an alpha tint (`--af-highlight-bg: rgba(31, 95, 160, 0.07)`). This is the discipline that keeps the palette coherent: new UI needs zero new colors, and a rebrand would touch nine lines. (Current scheme: "Ocean Fresh — Amber Nav" — the `--af-green*` names are historical, kept for a minimal diff; they hold navy blue, not green.)

6.2 Dark mode for free

Because components only reference variables, dark mode is a single media query that redefines them:

```
@media (prefers-color-scheme: dark) {
  :root {
    --af-bg: #0d1420;
    --af-surface: #1a2636;
    --af-text: #e1e7ee;
    --af-green: #3f7fc4;      /* brighter — dark backgrounds eat saturation */
    --af-green-dark: #96bee6; /* note: *lighter* in dark mode — it means "emphasis" */
    --af-error-text: #ffab70;
    /* ... */
  }
}
```

`prefers-color-scheme` follows the OS setting — no toggle, no JavaScript, no storage. Two details worth noticing: dark-mode greens are *brighter* (dark surfaces mute color, so you compensate), and `--af-green-dark` is actually lighter than `--af-green` in dark mode — the token names a *role* (emphasis), not a literal darkness. This is why components must never "correct" a color locally: any hardcoded hex is guaranteed wrong in one of the two schemes.

6.3 The rules

These are project law (see `CLAUDE.md` and `docs/design-rules.html`):

1. **One accent hue + amber highlight.** No new hues beyond the core `--af-green*` pair (currently navy) and `--af-amber`. If a design feels like it needs another color, it needs a different design.
2. **No gradients in UI.** The logo (`src/components/Logo.jsx`, `public/logo.svg` — the green shelf-"A" monogram) is the single exception.
3. **Never hardcode a hex in a component.** Use a `--af-*` variable; if no existing token fits, derive one in the shell's theme block (alias or alpha tint of the core nine) — in both schemes.
4. **Font Awesome icons, never emojis.** Icons come from a Font Awesome Kit script loaded in `public/index.html`. Use older icon names — `fa-cog`, not `fa-gear` — because the kit may not include newer aliases (a missing alias renders as blank space, not an error).
5. **Font is Arial, set once** on the shell in `AisleFinder.jsx`. Components use `fontFamily: 'inherit'` or nothing at all.

6.4 Component idioms

The styling itself is inline `style` objects (no CSS framework), with a few established patterns:

Hover effects are inline handlers, since inline styles can't express `:hover`:

```
<button
  onMouseEnter={e => { e.currentTarget.style.background = 'var(--af-green-dark)'; }}
  onMouseLeave={e => { e.currentTarget.style.background = 'var(--af-green)'; }}
>
```

Use `e.currentTarget` (the element the handler is attached to), **not** `e.target` — on a button containing an icon and text, `e.target` is whichever child the mouse touched, and the hover style lands on the wrong element.

Keyframe animations are injected with a plain `<style>` tag in the shell. Not `<style jsx>` — that's a Next.js feature, and this is CRA; the `jsx` attribute is silently ignored and the styles break in confusing ways.

Buttons need visible affordances — icon-only buttons have shipped looking invisible before; give buttons a text label and real padding.

6.5 Keeping it enforced

The system only works if every change goes through it. When you add UI:

- grep your diff for `#` colors before committing — the only hexes in `src/` should be in the shell's theme block;
- check both schemes (macOS: System Settings → Appearance, or the browser devtools emulation toggle);
- if you touched the theme block, update `docs/design-rules.html` to match.

TODOs when adding new UI

- ☐ **Open the visual reference** — open `docs/design-rules.html`
- ☐ **Pick tokens, not colors** — reuse an existing `--af-*` var; derive a new one in both light and dark blocks only if truly needed
- ☐ **Icons**: Font Awesome classic names only (`fa-cog` not `fa-gear`); no emojis anywhere in the UI
- ☐ **Hover**: `onMouseEnter` / `onMouseLeave` with `e.currentTarget`
- ☐ **Verify in dark mode** before calling it done
- ☐ **Sync** `docs/design-rules.html` if the theme block changed

07 Web Deployment

Locally you run two processes (Flask + CRA dev server). In production, one platform serves both: **Vercel** hosts the built React app as static files *and* runs the Python API as serverless functions, all behind one domain — <https://aislefinder3000.com>.

7.1 What Vercel is

Vercel is a hosting platform built around a simple loop:

1. You connect a Git repository.
2. Every `git push` triggers a **build** (here: `npm run build`).
3. The output is deployed globally to a CDN — static files are served from servers near the user.
4. Anything under `api/` becomes a **serverless function**: code that isn't running anywhere until a request arrives, at which point Vercel spins it up, handles the request, and tears it down. You pay per invocation and never manage a server.

Pushes to `master` deploy to production; pushes to any other branch get their own **preview URL** — a throwaway deployment for testing before merge.

7.2 How this repo is wired: `vercel.json`

```
{
  "buildCommand": "npm run build",
  "outputDirectory": "build",
  "framework": "create-react-app",
  "functions": {
    "api/index.py": { "runtime": "@vercel/python@4.5.0" }
  },
  "rewrites": [
    { "source": "/api/(.*)", "destination": "/api/index" },
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

Reading it top to bottom:

- **Build** — Vercel runs `npm run build` and serves the `build/` folder as static files (the React app).
- **Functions** — `api/index.py` runs on Vercel's Python runtime. It's a thin mirror of `api_server.py`: both register the same shared blueprints (`grocery_routes.py` , `lists_backend.py`), so local and production route logic literally cannot drift. If you edit one entry point, check the other.
- **Rewrites** — `/api/*` requests are routed to the Python function; *everything else* falls through to `index.html` so the React app loads no matter the URL (client-side routing).

The upshot: frontend and API share one origin in production, so there's no CORS complexity and `REACT_APP_API_URL` can simply be the site's own domain.

7.3 Connecting the repo to Vercel

```
# Option A: the dashboard (recommended first time)
# 1. Sign up at https://vercel.com (log in with GitHub)
# 2. "Add New... → Project" → import the aislefinder repo
# 3. Vercel auto-detects CRA; vercel.json supplies the rest → Deploy

# Option B: the CLI
# Run this
npm i -g vercel
vercel          # link the project + create a preview deployment
vercel --prod   # deploy to production
```

Then set the **environment variables** in the Vercel dashboard (*Project* → *Settings* → *Environment Variables*). The build and the Python function each read different ones:

VARIABLE	USED BY	PURPOSE
KROGER_CLIENT_ID	Python function	Kroger API auth (chapter 2)
KROGER_CLIENT_SECRET	Python function	Kroger API auth (chapter 2)
REACT_APP_COGNITO_USER_POOL_ID , REACT_APP_COGNITO_CLIENT_ID	React build	Login UI (chapter 10)
AISLEFINDER_TABLE , COGNITO_REGION	Python function	List sync (chapter 10)
AF_AWS_ACCESS_KEY_ID , AF_AWS_SECRET_ACCESS_KEY	Python function	AWS credentials — Vercel reserves the <code>AWS_*</code> names, hence the <code>AF_</code> prefix

After changing env vars, **redeploy** — `REACT_APP_*` values are baked in at build time.

7.4 The domain manager

A deployment gets a free `*.vercel.app` URL, but a real product wants a real domain. Three pieces are involved:

- **Registrar** — where you buy/renew the domain name (e.g. Namecheap, Cloudflare, or Vercel itself). This is a yearly fee (~\$10–15).
- **DNS** — the phone book mapping `aislefinder3000.com` → server addresses. Whoever hosts your DNS records controls where the name points.
- **Vercel's domain manager** (*Project* → *Settings* → *Domains*) — where you attach the domain to the project. Vercel provisions the HTTPS certificate automatically once DNS points at it.

7.5 How to connect the domain

1. In Vercel: *Project* → *Settings* → *Domains* → *Add* → enter `aislefinder3000.com`. Vercel shows you exactly which DNS records it needs.
2. At your registrar, either:
 - **Point the nameservers at Vercel** (`ns1.vercel-dns.com` , `ns2.vercel-dns.com`) — Vercel then manages all DNS for you, or
 - **Add individual records**: an `A` record for the apex (`@` → `76.76.21.21`) and a `CNAME` for `www` → `cname.vercel-dns.com`.

3. Wait for DNS propagation (minutes to a few hours). Vercel's Domains page flips to a green checkmark and issues the TLS certificate.

4. Verify:

```
# Run this
curl https://aislefinder3000.com/api/health
# → {"status": "healthy"}
```

Note the domain also appears in two places in the code: the CORS allowlist in `api_server.py:29` and the `ios:build` / `android:build` scripts in `package.json` (the mobile apps call the production API). If you ever change domains, update both.

TODOs to get this working

- ☐ **Create a Vercel account** at <https://vercel.com> (sign in with GitHub)
- ☐ **Push the repo to GitHub** if it isn't already, then **import it** as a Vercel project (Add New → Project)
- ☐ **Type the env vars into Vercel** — at minimum `KROGER_CLIENT_ID` and `KROGER_CLIENT_SECRET` ; the AWS ones come in chapter 10
- ☐ **Register the domain** (e.g. `aislefinder3000.com`) at a registrar
- ☐ **Add the domain in Vercel** (Settings → Domains) and **enter the DNS records / nameservers shown** at your registrar
- ☐ **Redeploy and verify** — `curl https://<your-domain>/api/health`

08 Adding iOS Support

Aisle Finder runs on iPhone without a rewrite, using **Capacitor**: the same React build ships inside a native iOS app.

8.1 How Capacitor works

Capacitor wraps your web app in a native shell. The iOS app is a real Xcode project whose main screen is a full-size **WKWebView** loading your `build/` folder from disk (served at the `capacitor://localhost` origin — which is why that origin is in the Flask CORS allowlist). When the web code needs something native — storage, camera, push — it calls a Capacitor **plugin** that bridges JavaScript to Swift.

The whole configuration is three lines (`capacitor.config.json`):

```
{
  "appId": "com.aislefinder.app",
  "appName": "Aisle Finder",
  "webDir": "build"
}
```

- `appId` — the reverse-DNS bundle identifier; this is your app's permanent identity in the App Store. Don't change it after shipping.
- `webDir` — which folder gets copied into the native app (CRA's `build/`).

The `ios/` directory is the generated Xcode project. It's checked into git — it contains real native config (Info.plist, icons, signing settings) that you edit over time. It was created once with `npm run ios`; you won't need to run that again.

8.2 The build loop

The key mental model: **the native app has a copy of your web build, not a live link to it**. After any web change, you rebuild and re-sync:

```
"ios:build": "REACT_APP_API_URL=https://aislefinder3000.com react-scripts build && cap sync ios"
"ios:open": "cap open ios"
```

```
# Run this — after any change to src/
npm run ios:build # build React for production, copy into ios/, update plugins
npm run ios:open  # open the project in Xcode
```

Two things in `ios:build` worth understanding:

- `REACT_APP_API_URL=https://aislefinder3000.com` — the mobile app can't use relative `/api/...` URLs like the website does (there's no web server inside the app), so the API base is baked in as the production domain. **This means the mobile apps depend on chapter 7 being done.**

- `cap sync ios` — copies `build/` into the iOS project and installs/updates any native plugin code (via CocoaPods).

Then in Xcode: pick a simulator (or your plugged-in iPhone) from the device dropdown and press ► **Run** (Cmd-R).

8.3 Native storage: why `storage.js` exists

iOS can evict a `WKWebView`'s `localStorage` when the device is low on space — which would silently delete a user's grocery lists. `src/storage.js` guards against this with the Capacitor **Preferences** plugin: every `localStorage` write is mirrored (debounced) to native storage (`UserDefaults` on iOS), and restored on launch if `localStorage` comes up empty. This is a good example of the pattern for platform-specific behavior: the web code checks `Capacitor.isNativePlatform()` and stays a no-op in the browser.

8.4 Signing and running on a real device

Running on the **simulator** needs nothing but Xcode. Running on a **real iPhone** requires code signing:

1. In Xcode, select the `App` target → *Signing & Capabilities*.
2. Check *Automatically manage signing* and pick your **Team** (your Apple ID — a free account works for on-device development; builds expire after 7 days).
3. Plug in the phone, select it as the run target, press Run. First time: on the phone, trust the developer cert under *Settings* → *General* → *VPN & Device Management*.

Distributing through the **App Store** additionally requires the paid Apple Developer Program (\$99/year), an app record in App Store Connect matching `com.aislefinder.app`, icons/screenshots, and an archive upload (*Product* → *Archive* → *Distribute App* in Xcode).

TODOs to get this working

- ☐ **A Mac** — iOS builds only work on macOS
- ☐ **Install Xcode** from the Mac App Store (large download), then run it once to accept the license and install the iOS platform
- ☐ **Install CocoaPods** (Capacitor's iOS dependency manager) — `brew install cocoapods`
- ☐ **Deploy the web app first** (chapter 7) — the app calls the production API baked in via `REACT_APP_API_URL`
- ☐ **Build + open** — `npm run ios:build && npm run ios:open`, then Run on a simulator
- ☐ **For a real device**: sign in with your Apple ID in Xcode → Settings → Accounts, set the Team under Signing & Capabilities
- ☐ **For the App Store**: enroll in the Apple Developer Program (\$99/year) at <https://developer.apple.com>, create the app record in App Store Connect, archive and upload from Xcode

09 Adding Android Support

Android support mirrors iOS (chapter 8) — same Capacitor model, different toolchain. Read chapter 8 first; this chapter covers what's different.

9.1 The Android flavor of Capacitor

The `android/` directory is a generated **Gradle** project (Android's build system, the counterpart of Xcode's project). The app is one Activity hosting a full-screen **WebView** that loads the copied `build/` folder from the `http://localhost / https://localhost` origin — also present in the Flask CORS allowlist. Plugins bridge JavaScript to Kotlin/Java; the same `storage.js` Preferences mirroring from chapter 8 lands in Android's `SharedPreferences`.

The same `capacitor.config.json` drives both platforms — `com.aislefinder.app` becomes the Android **applicationId** (the permanent identity on Google Play).

Key files inside `android/` you may eventually touch:

FILE	WHAT IT CONTROLS
<code>app/build.gradle</code>	applicationId, versionCode/versionName, SDK levels
<code>app/src/main/AndroidManifest.xml</code>	permissions, app name, deep links
<code>app/src/main/res/</code>	icons, splash screens
<code>variables.gradle</code>	shared SDK/dependency versions

9.2 The build loop

Identical shape to iOS:

```
"android:build": "REACT_APP_API_URL=https://aislefinder3000.com react-scripts build && cap sync"
"android:open": "cap open android"
```

```
# Run this – after any change to src/
npm run android:build # build React, copy into android/, sync plugins
npm run android:open  # open the project in Android Studio
```

In Android Studio: let Gradle finish syncing (progress bar at the bottom), create a virtual device if you don't have one (*Device Manager* → *Create Device*), then press ► **Run**.

Real devices are easier than iOS: enable Developer Options on the phone (tap *Settings* → *About phone* → *Build number* seven times), turn on **USB debugging**, plug in, and it appears in the run target dropdown — no signing account needed for development (Android auto-signs with a local debug key).

Command-line alternative if you don't want the IDE open:

```
# Run this – from android/  
./gradlew assembleDebug # produces app/build/outputs/apk/debug/app-debug.apk
```

9.3 Release signing

For the Play Store you sign with your own key instead of the debug key:

```
# Run this – one time; keep the file and passwords somewhere safe!  
keytool -genkey -v -keystore aislefinder-release.keystore \  
-alias aislefinder -keyalg RSA -keysize 2048 -validity 10000
```

Wire it into `android/app/build.gradle` as a `signingConfig` for the `release` build type (keep the keystore and its passwords **out of git** — reference them from `~/gradle/gradle.properties` or environment variables). Then build the upload bundle:

```
# Run this – from android/  
./gradlew bundleRelease # produces app/build/outputs/bundle/release/app-release.aab
```

Losing the keystore historically meant losing the ability to update your app; with **Play App Signing** (the default for new apps), Google holds the final signing key and your keystore is just the upload key — still, back it up.

TODOs to get this working

- ☐ **Install Android Studio** from <https://developer.android.com/studio> (bundles the Android SDK, an emulator, and a compatible JDK)
- ☐ **First launch:** let the setup wizard install the SDK + platform tools, and create one virtual device in Device Manager
- ☐ **Deploy the web app first** (chapter 7) — same production-API baking as iOS
- ☐ **Build + open** — `npm run android:build && npm run android:open`, wait for Gradle sync, press Run
- ☐ **For a real device:** enable Developer Options + USB debugging on the phone
- ☐ **For the Play Store:** create a Google Play Console developer account (\$25 one-time) at <https://play.google.com/console>, generate a release keystore with `keytool` (back it up!), add the `signingConfig`, build the `.aab` with `./gradlew bundleRelease`, create the app listing, and upload

10 Adding AWS Logins

Everything so far works in **guest mode** — lists live in `localStorage` on one device. Accounts add cross-device sync and list sharing, built on two AWS services:

- **Amazon Cognito** — a managed user directory: sign-up, email verification, password auth, and token issuance. You never store passwords yourself.
- **DynamoDB** — a serverless key-value database holding lists, memberships, and share codes. No server to run; you pay per request.

A deliberate design property: if the AWS env vars are absent, `auth.js` exports `authConfigured = false`, the list endpoints return 503, and the app silently stays guest-only. Nothing breaks before setup — so this chapter is genuinely optional.

10.0 AWS for absolute beginners

If you've used AWS before, skip to [10.1](#).

What is AWS? Amazon Web Services is a collection of cloud infrastructure services. Instead of running your own database server or authentication system, you rent capacity from Amazon and pay for what you use. For this app at hobby scale, the monthly cost will likely be \$0 thanks to generous free tiers.

What is "the cloud" here? Concretely: computers in Amazon's data centers running software on your behalf. When a user signs up, their account gets created in Amazon's systems, not on your laptop. This means it works even when your laptop is off, and it scales automatically if you get more users.

The mental model for this chapter:

```

User's browser —sign-in→ Cognito (AWS)
                        (manages passwords, tokens)

User's browser —save list→ Your Flask server —stores→ DynamoDB (AWS)
  
```

You never touch passwords directly. Cognito handles all the cryptography. DynamoDB stores lists and syncs them across devices.

10.1 Create an AWS account

Go to <https://aws.amazon.com> and click **Create an AWS Account** in the top right.

You'll need:

- An **email address** (this becomes your "root" account — use a real one you can receive email at)

Also, a **phone number** for identity verification (they send an SMS or call you)

- A **credit card** — AWS requires one even for free-tier usage; you won't be charged for what this app uses at small scale

The sign-up steps

1. Enter your email and choose an account name (e.g. `hayley-personal` or `aislefinder-dev` — it's just a label).
2. Verify your email address via the code they send.
3. Choose **Personal** account type (not Business).
4. Fill in your contact information.
5. Add a credit card. AWS won't charge you for free-tier usage but needs one on file.
6. Verify your phone number via SMS or voice call.
7. Choose a **Support plan** — pick **Basic (Free)**. You don't need paid support.
8. Sign in to the AWS Console at <https://console.aws.amazon.com>.

The AWS Console

The Console is a web dashboard where you can see and manage everything you've created. When you first log in it looks overwhelming — there are hundreds of services. You only need two: **Cognito** and **DynamoDB**. You can find any service by typing its name in the search bar at the top.

Important: the root account vs IAM users

When you first create your AWS account, you sign in as the **root user** — an all-powerful account that can do anything, including delete everything and rack up a huge bill. **Never use the root account for day-to-day work.** If your root credentials are ever leaked, an attacker has unlimited access.

Instead, you create **IAM users** (Identity and Access Management) — accounts with specific, limited permissions. This chapter has you:

1. Create an **admin IAM user** for yourself — has enough access to create Cognito pools and DynamoDB tables, but isn't the root account.
2. Later, create a **minimal IAM user** for your production server — only has the exact permissions the running app needs.

This "principle of least privilege" limits damage if a credential leaks.

10.2 Create an IAM admin user for yourself

Why not just use the root account? The setup script needs to create Cognito and DynamoDB resources. You could use root credentials, but you'd be training yourself into a bad habit. An admin IAM user is nearly as capable and much safer to use daily.

Steps in the AWS Console

1. In the Console search bar, type **IAM** and click the IAM service.
2. In the left sidebar, click **Users**, then click **Create user**.
3. Enter a username — something like `hayley-admin` or `aislefinder-admin`. Do **not** check "Provide user access to the AWS Management Console" unless you specifically want this IAM user to log in to the Console (usually you don't need that for a developer IAM user).

4. Click **Next**. On the permissions page, choose **Attach policies directly**, then search for and select **AdministratorAccess**. This gives the user full access to all AWS services — broad but fine for your personal admin user.
5. Click **Next**, then **Create user**.
6. Click the username you just created to open it. Go to the **Security credentials** tab. Under **Access keys**, click **Create access key**.
7. Choose **Command Line Interface (CLI)** as the use case, check the confirmation box, and click **Next**.
8. Optionally add a description tag (e.g. `dev laptop`), then click **Create access key**.
9. **This is your only chance to see the Secret Access Key.** Copy both the **Access key ID** and **Secret access key** somewhere safe (a password manager works well). Click **Done**.

You'll paste these into `aws configure` in the next section.

10.3 Install and configure the AWS CLI

The AWS CLI is a command-line tool that lets scripts and your terminal talk to AWS. The setup script (`infra/setup-aws.sh`) uses it to create the Cognito pool and DynamoDB table.

Install

macOS (Homebrew):

```
brew install awscli
```

macOS (without Homebrew) / other platforms:

Download the installer from the [AWS CLI install page](#).

Verify it installed:

```
aws --version
# Should print something like: aws-cli/2.x.x ...
```

Configure with your IAM credentials

```
aws configure
```

It asks four questions:

```
AWS Access Key ID [None]: AKIA...      ← paste your access key ID
AWS Secret Access Key [None]: abc123... ← paste your secret access key
Default region name [None]: us-east-1  ← type this exactly
Default output format [None]: json     ← type this
```

Region is which Amazon data center your resources live in. `us-east-1` (Northern Virginia) is the most common choice and has the best service availability. Pick one and stick with it — all resources in this project should be in the same region.

These credentials get saved in `~/.aws/credentials` on your laptop. The CLI reads them automatically; you never pass them as command-line arguments.

Verify it works

```
aws sts get-caller-identity
```

Expected output (your account ID will differ):

```
{
  "UserId": "AIDA...",
  "Account": "123456789012",
  "Arn": "arn:aws:iam::123456789012:user/hayley-admin"
}
```

If you get an error like `InvalidClientTokenId`, double-check that you pasted the keys correctly and that the key hasn't been deactivated.

10.4 What the setup script creates

`infra/setup-aws.sh` is an **idempotent** script — safe to run multiple times. If a resource already exists it skips creation instead of failing.

Run it with the region you chose during `aws configure`:

```
./infra/setup-aws.sh us-east-1
```

It creates three things:

1. Cognito User Pool: `aislefinder-users`

A **user pool** is Cognito's name for a directory of users. Configuration:

- **Email as username** — users sign up and log in with their email address, not a separate username.
- **Email verification** — when a user signs up, Cognito emails them a 6-digit code they must enter to confirm their address. This proves email ownership without you writing any email-sending code.
- **8-character minimum password** — the pool enforces this automatically.
- **No admin-created users** — users self-register through the app's sign-up form; you don't provision accounts manually.

2. App Client: `aislefinder-web`

Within a user pool, an **app client** represents one application that talks to Cognito. Configuration:

- **No client secret** — browser apps can't keep secrets (anyone can open DevTools and read them), so this client is configured as a public client.
- **SRP auth only** — Secure Remote Password is a challenge-response protocol where the password is never sent over the network in any form, even encrypted. The browser proves it knows the password without revealing it.

3. DynamoDB table: `aislefinder-lists`

A **DynamoDB table** is like a key-value store crossed with a document database. No columns to define upfront — each item can have different fields. Configuration:

- **On-demand billing** — you pay per read/write operation rather than reserving capacity. At hobby scale this is nearly free.
- **byShareCode global secondary index (GSI)** — an index that lets the app quickly find a list by its share code without scanning the whole table. Like an index in a relational database.

What the script prints

When it finishes, the script prints four values:

```
REACT_APP_COGNITO_USER_POOL_ID=us-east-1_XXXXXX
REACT_APP_COGNITO_CLIENT_ID=1abc2def3ghi4jkl5mno6pqr
AISLEFINDER_TABLE=aislefinder-lists
COGNITO_REGION=us-east-1
```

Copy all four — you'll need them in the next section.

10.5 Wire the environment variables

Local development (`.env` file)

Open `.env` in the repo root (create it from `.env.example` if it doesn't exist). Add the four values the script printed:

```
REACT_APP_COGNITO_USER_POOL_ID=us-east-1_XXXXXX
REACT_APP_COGNITO_CLIENT_ID=1abc2def3ghi4jkl5mno6pqr
AISLEFINDER_TABLE=aislefinder-lists
COGNITO_REGION=us-east-1
```

Why two pairs? The `REACT_APP_*` vars are read by the React build and baked into the JavaScript bundle served to browsers. The other two are read by the Flask server at runtime. The split is intentional — the frontend talks to Cognito directly; the backend talks to DynamoDB and validates tokens.

Restart both dev servers after editing `.env` :

```
# In one terminal:
npm start

# In another terminal:
python api_server.py
```

The React dev server only reads `REACT_APP_*` at startup time — changes to `.env` don't hot-reload.

Local AWS access: The Flask server uses `boto3` (the Python AWS SDK) to talk to DynamoDB. Locally, `boto3` automatically reads your `~/.aws/credentials` file set by `aws configure` — you don't need to add AWS credentials to `.env` for local development.

Test that it's working locally

1. Start both servers.

2. Open the app in your browser.

3. Tap the account icon (top right).
4. The sheet should show a **Sign Up** form rather than a "sign in unavailable" message.
5. Sign up with an email address you can receive mail at.
6. Check your inbox — Cognito sends a verification code within a minute or two. (Check spam if it doesn't arrive.)
7. Enter the code in the app. You should be signed in.
8. Open the app in a second browser or incognito window, sign in with the same credentials, and verify your lists appear on both.

10.6 How the auth flow works (code walkthrough)

You don't need to understand this to get it running, but it's useful context for debugging.

Sign-in: frontend to Cognito (no server involved)

`src/auth.js` talks to Cognito directly via `amazon-cognito-identity-js`. The Flask server is not involved in the sign-in step:

```
export const signIn = (email, password) =>
  new Promise((resolve, reject) => {
    const cognitoUser = new CognitoUser({ Username: email, Pool: pool });
    const details = new AuthenticationDetails({ Username: email, Password: password });
    cognitoUser.authenticateUser(details, {
      onSuccess: (session) => resolve(sessionToUser(cognitoUser, session)),
      onFailure: (err) => reject(err),
    });
  });
```

Sign-up is a three-step sequence you can watch in `AccountSheet.jsx`:

```
signUp(email, password)
  → Cognito emails a 6-digit verification code
    → confirmSignUp(email, code)
      → signIn(email, password)
```

JWTs: what Cognito gives back after sign-in

On success Cognito issues **JWTs** (JSON Web Tokens) — cryptographically signed blobs of data. Two tokens matter:

- **ID token** — contains who you are: your `email` and `sub` (a stable UUID that never changes even if you update your email). `auth.js` reads these from the token payload.
- **Access token** — proof you're logged in, used for API calls. The SDK stores both tokens in `localStorage` and automatically refreshes them before they expire (default: 1-hour expiry). `getAccessToken()` in `auth.js` returns the current valid token.

Authenticated requests: frontend to Flask

Every API call that needs auth attaches the token as a Bearer header (`src/api.js`):

```
const jsonHeaders = (token) => ({
  'Content-Type': 'application/json',
  ...(token ? { Authorization: `Bearer ${token}` } : {}),
});
```

Token validation: Flask to Cognito

`lists_backend.py` receives the bearer token, calls Cognito's `GetUser` API to validate it (Cognito rejects expired or tampered tokens), and extracts the user's `sub`. Only then does it touch DynamoDB.

This validation is cached for ~5 minutes per token to avoid a Cognito round trip on every single request. The `sub` is the stable user identity used as DynamoDB's partition key.

10.7 Database schema: what's stored in DynamoDB

Everything lives in **one table** (`aislefinder-lists`) using "single-table design" — instead of one table per entity type (the relational instinct), different kinds of items share a table and are told apart by the shape of their partition key (`pk`) and sort key (`sk`). This is the standard DynamoDB pattern: it keeps related items in the same partition so they can be fetched together, and it means one table to provision, back up, and pay for.

Table definition

SETTING	VALUE
Partition key (<code>pk</code>)	String
Sort key (<code>sk</code>)	String
Billing mode	On-demand (pay per request, no reserved capacity)
Global secondary index	<code>byShareCode</code> , partition key <code>share_code</code> (String), projects all attributes
Item size limit	400 KB (DynamoDB's hard cap). The app enforces a tighter 256 KB (<code>MAX_LIST_BYTES</code> in <code>lists_backend.py</code>) on incoming list bodies, leaving headroom for the server-owned fields added on top.

There are no other tables. Everything — lists, memberships, sharing, and the aisle-correction feature described below — lives in these rows, distinguished only by `pk` / `sk` prefix.

Item shapes

Four kinds of item share the table:

PK	SK	WHAT IT IS
<code>LIST#{id}</code>	<code>META</code>	The full list JSON, plus members map and <code>share_code</code>
<code>USER#{sub}</code>	<code>LIST#{id}</code>	Membership marker: "this user can see this list"
<code>AISLE#{storeId}#{itemKey}</code>	<code>USER#{sub}</code>	One user's aisle/category correction for one item at one store
<code>AISLE#{storeId}#{itemKey}</code>	<code>CONSENSUS</code>	Derived: the community-agreed placement for that item, once enough shoppers agree

List record — `pk=LIST#{id}` , `sk=META`

ATTRIBUTE	TYPE	MEANING
<code>data</code>	String	The list itself, JSON-encoded (client payload with <code>members</code> / <code>shareCode</code> stripped — see Security below)
<code>owner_sub</code>	String	Cognito <code>sub</code> of whoever created the list; only the owner can delete it for everyone
<code>members</code>	Map (<code>sub</code> → String)	Every member's <code>sub</code> mapped to a display name (the part of their email before <code>@</code>)
<code>share_code</code>	String, optional	The 6-character join code, once one has been generated; absent until <code>POST /api/lists/:id/share</code> is called
<code>updated_at</code>	String	Client-supplied timestamp from the list's <code>updatedAt</code> field

Membership marker — `pk=USER#{sub}` , `sk=LIST#{id}`

No attributes beyond the key itself — its only job is to answer "which lists can this user see," via a `Query` on `pk = USER#{sub}` . One item exists per (user, list) pair; leaving a list or deleting a list removes it.

Aisle-override vote — `pk=AISLE#{storeId}#{itemKey}` , `sk=USER#{sub}`

One user's correction to where an item lives at a specific store — e.g. "this store actually shelves oat milk in aisle 4, not dairy." `itemKey` is the grocery item name, lowercased and trimmed, so votes for "Milk" and "milk" land on the same partition.

ATTRIBUTE	TYPE	MEANING
<code>kind</code>	String	<code>'aisle'</code> , <code>'category'</code> , or <code>'none'</code> (an opinion that the store doesn't carry the item)
<code>value</code>	Number, String, or null	Aisle number for <code>kind='aisle'</code> , category name for <code>kind='category'</code> , null for <code>kind='none'</code>
<code>store_aisle</code>	Number or null	The store's own (uncorrected) aisle number at the time of voting, for context
<code>item</code>	String	The item name as originally typed (display casing, unlike <code>itemKey</code>)
<code>updated_at</code>	String	ISO-ish timestamp (<code>_now_iso()</code>) of the last time this user voted

Sending `placement: null` to `PUT /api/aisle-overrides` deletes this item instead of writing one — that's how a user clears their own correction.

Aisle-override consensus — `pk=AISLE#{storeId}#{itemKey}` , `sk=CONSENSUS`

A derived row, not written directly by any client — recomputed inline by `_recompute_consensus()` every time a vote in that partition changes, by querying all `USER#` sibling rows and tallying agreement. No DynamoDB Streams or Lambda involved; it's just a second write in the same request.

ATTRIBUTE	TYPE	MEANING
<code>kind</code> / <code>value</code>	same as vote rows	The most-agreed-on placement
<code>agree</code>	Number	How many distinct users voted for that exact placement
<code>voters</code>	Number	Total distinct users who have voted on this item at this store (any placement)
<code>updated_at</code>	String	Timestamp of the last recompute

The row only exists once `agree >= CONSENSUS_MIN_AGREE` (currently 3) — below that threshold it's deleted rather than left stale, so "no consensus row" always means "not enough agreement yet."

Access patterns

Loading your lists: `GET /api/lists` queries all `USER#{sub}` items to find which lists you're a member of, then fetches each list's `META` item. Two DynamoDB calls total, regardless of list count.

Sharing: Sharing generates a 6-character code (skipping ambiguous characters like 0/O and 1/I/L) stored on the list's `META` item. `POST /api/lists/join` looks up that code via the `byShareCode` GSI and adds the joiner as a member by writing a `USER#{sub}` item.

Resolving aisle overrides: `POST /api/aisle-overrides/resolve` looks up, per item, everything under one `AISLE#{storeId}#{itemKey}` partition in a single `Query` — the caller's own vote, any household (shared-list member) votes, and the `CONSENSUS` row — then picks the highest-precedence one: mine, then household, then community. This is why votes and consensus share a partition key instead of living in separate lookups.

Security: The members map and share code on list records are server-owned. `PUT /api/lists` strips them from client payloads, so a client can't grant itself access to other lists by crafting a request. Likewise, `resolve_aisle_overrides` only returns a household member's vote if the caller is already a verified member of the list naming that household — you can't probe another user's corrections by guessing a `listId`.

10.8 Production setup: Vercel and a minimal IAM user

Local development uses your admin IAM credentials automatically. Production (Vercel) is a different story — Vercel's servers have no AWS identity, so you must provide credentials explicitly. But you should **never use your admin keys in production** — if they leaked, an attacker could do anything.

Instead, create a second IAM user with only the permissions the running app actually needs.

Create the minimal IAM user

1. In the AWS Console, go to **IAM** → **Users** → **Create user**.
2. Name it `aislefinder-prod` (or similar).
3. On permissions: choose **Attach policies directly**. Do **not** add AdministratorAccess. Instead click **Create policy** to open the policy editor.
4. Switch to the **JSON** tab and paste this policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "dynamodb:GetItem",
        "dynamodb:PutItem",
        "dynamodb:UpdateItem",
        "dynamodb>DeleteItem",
        "dynamodb:Query",
        "dynamodb:BatchWriteItem"
      ],
      "Resource": [
        "arn:aws:dynamodb:us-east-1:YOUR_ACCOUNT_ID:table/aislefinder-lists",
        "arn:aws:dynamodb:us-east-1:YOUR_ACCOUNT_ID:table/aislefinder-lists/index/byShareCode"
      ]
    },
    {
      "Effect": "Allow",
      "Action": "cognito-idp:GetUser",
      "Resource": "arn:aws:cognito-idp:us-east-1:YOUR_ACCOUNT_ID:userpool/us-east-1_XXXXXXX"
    }
  ]
}
```

Replace `YOUR_ACCOUNT_ID` with your 12-digit AWS account ID (visible in the top-right of the Console under your username) and `us-east-1_XXXXXXX` with your actual user pool ID from the script output.

5. Name the policy `aislefinder-prod-policy`, create it, then attach it to the `aislefinder-prod` user.
6. After creating the user, go to **Security credentials** → **Create access key**, choose **Other** as the use case, and save the key pair.

Add vars to Vercel

In your Vercel project dashboard, go to **Settings** → **Environment Variables**. Add all six:

VARIABLE	VALUE
<code>REACT_APP_COGNITO_USER_POOL_ID</code>	from script output
<code>REACT_APP_COGNITO_CLIENT_ID</code>	from script output
<code>AISLEFINDER_TABLE</code>	<code>aislefinder-lists</code>
<code>COGNITO_REGION</code>	<code>us-east-1</code>
<code>AF_AWS_ACCESS_KEY_ID</code>	from the <code>aislefinder-prod</code> IAM user
<code>AF_AWS_SECRET_ACCESS_KEY</code>	from the <code>aislefinder-prod</code> IAM user

Why `AF_` prefix? Vercel reserves the `AWS_*` namespace for its own internal use. The app reads `AF_AWS_ACCESS_KEY_ID` and `AF_AWS_SECRET_ACCESS_KEY` specifically to work around this.

After saving, trigger a redeploy (Vercel → Deployments → Redeploy). The `REACT_APP_*` vars are baked into the JS bundle at build time, so a redeploy is required even if only those values changed.

10.9 Mobile builds

The two `REACT_APP_COGNITO_*` vars must be present in your local shell environment when you build the mobile apps — they get baked into the JS bundle that Capacitor packages:

```
# Make sure your .env has the vars, then:
npm run ios:build
# or
npm run android:build
```

If you build without them, the app will ship in guest-only mode even on devices where auth would otherwise work. Re-build and re-submit to fix it.

10.10 Costs

At hobby scale this is effectively free:

SERVICE	FREE TIER	WHAT YOU'LL ACTUALLY USE
Cognito	10,000 monthly active users	likely single digits
DynamoDB	25 GB storage + 200M requests/month	kilobytes, hundreds of requests
Data transfer	1 GB/month outbound	negligible

Your bill will realistically be **\$0.00/month** until you have hundreds of active users. DynamoDB on-demand pricing is \$1.25 per million write request units and \$0.25 per million read request units — at personal-project scale you won't hit even a million requests in a month.

Set up a **billing alert** so you're never surprised: AWS Console → Billing → Budgets → Create budget → Zero spend budget. AWS will email you the moment any charge appears.

10.11 Troubleshooting

"Sign up" form doesn't appear — just shows a message about auth being unavailable

The `REACT_APP_COGNITO_*` env vars aren't set or the dev server wasn't restarted after setting them. Check:

```
# Should print non-empty values:
echo $REACT_APP_COGNITO_USER_POOL_ID
echo $REACT_APP_COGNITO_CLIENT_ID
```

If those are empty, your `.env` file isn't being picked up. Make sure `.env` is in the repo root (same directory as `package.json`), not a subdirectory.

Verification code email never arrives

- Check spam/junk.
- Wait up to 5 minutes — Cognito sends via SES and there can be a delay.
- Make sure you entered the correct email during sign-up; no typos.
- You can resend the code: in `AccountSheet.jsx` there's a resend link during the confirmation step.

Lists don't sync to the second device

- Confirm you're signed in on both devices (account icon should show your email, not a "Sign in" prompt).
- Wait ~15 seconds after saving on the first device — the sync has a debounce and polling interval.
- Open the browser console (F12 → Console) and look for red errors, especially 401/403 HTTP errors which indicate a token problem, or 503 which means the backend couldn't reach DynamoDB.

aws configure errors when running the setup script

```
Error: InvalidClientId
```

The access key ID is wrong — check for copy-paste errors.

```
Error: SignatureDoesNotMatch
```

The secret access key is wrong or contains extra whitespace.

```
Error: AuthFailure – not authorized
```

The IAM user doesn't have AdministratorAccess. Go back to IAM and check the attached policies.

Script fails with "already exists" errors

The script is idempotent but may fail loudly on partial setups. Check what already exists in the Console (Cognito → User Pools, DynamoDB → Tables) and either delete the partial resources and rerun, or manually copy the IDs of the existing resources into your `.env`.

TODOs checklist

- ☐ **Create an AWS account** at <https://aws.amazon.com> (needs a credit card; this setup stays in the free tier — set up a zero-spend billing alert too)
- ☐ **Create an admin IAM user** (not the root account), generate an access key, and run `aws configure` with it
- ☐ **Verify the CLI works:** `aws sts get-caller-identity` should return your account ID
- ☐ **Run the setup script:** `./infra/setup-aws.sh us-east-1`
- ☐ **Copy the four printed env values into `.env`**, restart both dev servers (`npm start` + `python api_server.py`)
- ☐ **Test locally:** sign up in the account sheet, receive the email code, confirm, sign in — make sure no errors appear in the browser console
- ☐ **Test sync:** open the app in two browsers, add an item in one, watch it appear in the other within ~15 seconds
- ☐ **For production:** create the minimal `aislefinder-prod` IAM user with the restricted policy, add all six vars to Vercel, redeploy
- ☐ **For mobile:** make sure `REACT_APP_COGNITO_*` vars are in your environment when running `npm run ios:build` / `android:build`

11 Offline-First Sync

The subtlest code in the app is `src/listsStore.js` — one hook that keeps grocery lists correct across offline edits, multiple devices, and shared lists, without a sync library. This chapter dissects it. (Chapter 10 covered the server side; this is the client half.)

Offline-first means the local copy is the source of truth for the UI: every edit applies instantly to local state and `localStorage`, and the network is only ever catching up in the background. The app never shows a spinner for a list edit.

11.1 The data shape

Everything revolves around one list object (`newList()`):

```
{
  id: uid(),
  name, status: 'active',      // or 'completed'
  items: [],                  // [{ id, name, addedAt, addedBy, fromList }]
  store: null,                // the picked Kroger store
  organized: null,            // markdown from the last organize call
  checkedItems: {}, collapsedGroups: {}, customCategoryOrder: null,
  shareCode: null, members: [], // server-owned (chapter 10)
  createdAt, updatedAt, completedAt,
}
```

Two fields do the sync heavy lifting: `updatedAt` (bumped on every mutation — the conflict tiebreaker) and `id` (generated client-side, so lists created offline need no server round trip to exist).

11.2 Local persistence: every render, every write

Persistence isn't an event — it's a `useEffect` that mirrors state whenever it changes:

```
const [lists, setLists] = useState(initialLists);
useEffect(() => { saveState('lists', lists); }, [lists]);
```

`saveState` (in `storage.js`) writes `localStorage` synchronously and, on iOS/Android, also mirrors the value to native storage via Capacitor Preferences (debounced ~100ms per key) so the OS can't evict it (chapter 8). On startup, `initialLists()` loads from `localStorage` — and falls back to migrating the pre-redesign single-list format, the path pinned down by the migration test in chapter 5.

11.3 Dirty tracking: remembering what the server hasn't seen

Every mutation marks its list **dirty** — "changed locally, not yet confirmed by the server":

```
const dirtyIds = useRef(new Set(loadState('dirtyListIds', [])));

const markDirty = useCallback((id) => {
  dirtyIds.current.add(id);
  saveState('dirtyListIds', [...dirtyIds.current]);
}, []);
```

Details that matter:

- It's a `useRef`, not `useState` — dirtiness is bookkeeping, not UI; it shouldn't cause re-renders.
- It's **persisted**. Close the app with unsynced edits and the dirty set survives the restart — nothing is lost, just late.
- It's a set of list IDs, not a diff. Sync always pushes the whole current list, which makes retries trivially safe (idempotent PUT).

11.4 Pushing: debounced, per-list, failure-tolerant

```
// Debounced push after any mutation
useEffect(() => {
  if (!user) return;
  clearTimeout(pushTimer.current);
  pushTimer.current = setTimeout(pushDirty, 1500);
  return () => clearTimeout(pushTimer.current);
}, [lists, user, pushDirty]);
```

Every state change resets a 1.5-second timer — type five items quickly and the app makes one request, not five.

Inside `pushDirty()`:

```
for (const id of [...dirtyIds.current]) {
  try {
    const saved = await pushList(token, list);
    dirtyIds.current.delete(id); // clean only after the server confirms
    if (saved && saved.members) { /* adopt server-owned members/shareCode */ }
  } catch { /* stays dirty; retried next cycle */ }
}
```

The empty `catch` is the whole failure strategy, and it's the right one: a failed push changes nothing — the list simply *stays dirty* and the next cycle retries. No error queues, no exponential backoff bookkeeping. Note also the write-back: `members` and `shareCode` come from the server's response, never from local guesses (they're server-owned, chapter 10).

11.5 Pulling: four triggers, one conflict rule

`pullRemote()` runs on sign-in, every 15 seconds while the tab is visible, on window focus, and when connectivity returns:

```
const onOnline = () => { pushDirty().then(pullRemote); }; // push first, then pull!
```

(Push-then-pull on reconnect matters: pulling first could overwrite offline edits that haven't been sent yet.)

Merging remote lists into local state is three lines of policy:
AisleFinder — Developer Guide

```

if (!local) {
  byId.set(r.id, r); // new from another device: take it
} else if (!dirtyIds.current.has(r.id) && (r.updatedAt || '') > (local.updatedAt || '')) {
  byId.set(r.id, r); // clean here + newer there: take it
}
// otherwise: keep local

```

In words: **a dirty local list always wins** (your unsent edits are never clobbered), and a clean local list defers to a newer server copy. This is last-write-wins at list granularity — simple, predictable, and honest about its trade-off: if two people edit the *same list* while both offline, the later sync wins whole. For a grocery list, that's an acceptable deal.

One more subtlety at the end of `pullRemote()`: any local list the server has never heard of gets marked dirty, so lists created before you signed in (or while the backend was down) upload themselves on the next cycle.

11.6 The complete picture

```

edit → setLists → useEffect → localStorage (+ native mirror)
      |
      └─ markDirty → 1.5s debounce → PUT /api/lists/{id} → clean
                                   ▲ (fail → stays dirty)
sign-in / 15s visible / focus / reconnect → pullRemote
      merge rule: dirty-local wins, else newer updatedAt wins

```

Guest mode is just this diagram with the bottom half switched off — which is why every sync effect starts with `if (!user) return;`

TODOs to verify it works

- ☐ **Two-device sync** — with chapter 10 configured, sign in on two browsers; an item added in one appears in the other within ~15s
- ☐ **Offline edits** — DevTools → Network → Offline; add items (works instantly), go back online, watch the push fire and the other browser catch up
- ☐ **Restart survival** — go offline, edit, fully close the tab, reopen: the edits and the pending dirty state are both still there
- ☐ **Guest-mode isolation** — sign out and confirm zero requests to `/api/lists*` in the Network tab

12 The Full Codebase

A map of every directory and how a request actually flows through the system.

12.1 The annotated tree

aislefinder/

└─ src/	— REACT FRONTEND (chapters 4–6)
└─ index.js	CRA entry – mounts <AisleFinder />
└─ AisleFinder.jsx	Shell: screen/sheet state, theme CSS vars, toasts
└─ listsStore.js	useLists() – data model + background sync (ch. 11)
└─ auth.js	useAuth() – Cognito sign-in (chapter 10)
└─ api.js	All fetch calls to the backend
└─ storage.js	localStorage + Capacitor Preferences mirror
└─ listUtils.js	Parse/build the markdown list format
└─ AisleFinder.test.js	Jest: app rendering + data migration (ch. 5)
└─ listUtils.test.js	Jest: pure markdown-parsing tests
└─ screens/	
└─ └─ CurrentListScreen.jsx	Home: quick-add, items, frequent suggestions
└─ └─ MyListsScreen.jsx	Switch / create / delete / merge lists
└─ └─ HistoryScreen.jsx	Completed trips grouped by month
└─ └─ ShopScreen.jsx	Aisle groups, drag-drop, check-off, confetti
└─ components/	
└─ └─ TopBar.jsx	Sticky header + nav buttons
└─ └─ Sheet.jsx	Bottom-sheet modal wrapper
└─ └─ AccountSheet.jsx	Sign in / sign up / confirm code
└─ └─ ShareSheet.jsx	Create + join share codes
└─ └─ StoreSheet.jsx	ZIP search → pick a store
└─ └─ Logo.jsx	The green shelf-"A" monogram (SVG)
└─ public/	— CRA static assets (index.html shell, logo.svg)
└─ api_server.py	— FLASK ENTRY, local dev (chapter 1)
└─ api/index.py	— FLASK ENTRY, Vercel serverless (chapter 7)
└─ grocery_routes.py	— Blueprint: product/aisle/store endpoints
└─ lists_backend.py	— Blueprint: Cognito auth + DynamoDB lists (ch. 10)
└─ grocery_organizer/	— PYTHON CORE (framework-free; no Flask imports)
└─ └─ main.py	CLI entry point (predates the web app)
└─ └─ tests/	pytest suite (chapter 3)
└─ └─ src/	
└─ └─ └─ core/	
└─ └─ └─ └─ processor.py	GroceryListProcessor – the orchestrator
└─ └─ └─ └─ └─ models.py	FullProduct dataclass (name, match, category, aisle)
└─ └─ └─ input_parsing/	
└─ └─ └─ └─ input_parser.py	Text → item list (newlines, commas, bullets)
└─ └─ └─ store_api/	
└─ └─ └─ └─ kroger.py	KrogerAPI: OAuth, product search + scoring (ch. 2)
└─ └─ └─ └─ search_terms.py	Search-term cleanup (quantities, filler words)
└─ └─ └─ output_formatting/	
└─ └─ └─ └─ output_formatter.py	Group + sort + render markdown
└─ ios/	— Capacitor Xcode project (chapter 8)
└─ android/	— Capacitor Gradle project (chapter 9)
└─ capacitor.config.json	— appId, appName, webDir
└─ infra/	
└─ └─ setup-aws.sh	Idempotent Cognito + DynamoDB creation (ch. 10)
└─ └─ README.md	AWS wiring notes
└─ vercel.json	— Build config + /api rewrites (chapter 7)
└─ package.json	— npm deps + build scripts
└─ requirements.txt	— Python deps
└─ .env	— Secrets (git-ignored; see .env.example)
└─ docs/	

└─ design-rules.html	Design tokens: colors, spacing, motion (ch. 6)
└─ dev-guide/	← you are here
└─ mockups/	— Design explorations
└─ api-response-viewer.html	— Local debug UI for raw Kroger responses (ch. 2)
└─ CLAUDE.md	— Codebase notes for AI-assisted development
└─ DEPLOYMENT.md	— Deployment quick-reference
└─ README.md	— Project overview

12.2 The three-layer backend

The Python side is deliberately layered so each piece is testable alone:

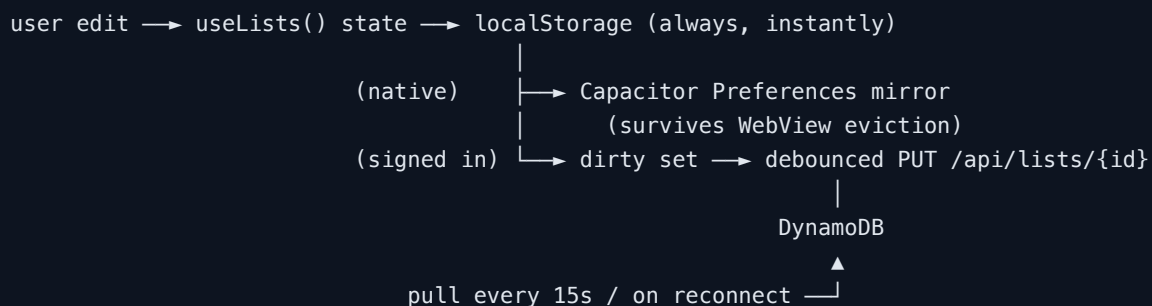
entry points	api_server.py (local)	api/index.py (Vercel)
	└─ \	└─ /
routes (blueprints)	grocery_routes.py	lists_backend.py
	└─	└─
domain logic	grocery_organizer/src/	boto3 → DynamoDB/Cognito

- **Entry points** contain no route logic — only app creation, CORS, and blueprint registration. This is the "two servers can't drift" rule: if you add an endpoint, it goes in a blueprint, never in an entry point.
- `grocery_organizer/src/` never imports Flask. It's a plain Python library, which is why the CLI (`main.py`) and the pytest suite can drive it directly.

12.3 Life of a request: "organize my list"

1. **ShopScreen** calls `processGroceryList()` in `src/api.js` → `POST /api/process-grocery-list` with the items and the chosen store's ID.
2. `grocery_routes.py` builds a `GroceryListProcessor` and calls `process_list()`.
3. **InputParser** normalizes the text into item strings.
4. The processor creates a **KrogerAPI** for the store, pre-fetches the OAuth token, and fans lookups out **in parallel** (`ThreadPoolExecutor`); each returns a `FullProduct(input_name, found_product, category, aisle_number)`. A failed lookup becomes a "Not Found" entry instead of failing the whole list.
5. **OutputFormatter** groups by aisle (falling back to category), sorts sections in a food-safe shopping order (produce first, frozen/dairy last), and renders markdown: `## AISLE 3\n- bread`.
6. The frontend's `listUtils.js` parses that markdown back into groups for ShopScreen to render as draggable, checkable sections.

12.4 Life of a list: where data lives



Guest mode is the baseline; everything below the first line is progressive enhancement that turns on only when configured (chapters 8–10). Chapter 11 walks this diagram in code.

12.5 Running everything

```
# Backend (venv active, .env in place)
python api_server.py                                # → localhost:8000

# Frontend
npm start                                           # → localhost:3000

# CLI (no web stack needed)
PYTHONPATH=. python grocery_organizer/main.py \
  --file=grocery_organizer/list.txt --output_format=aisle

# Tests
PYTHONPATH=. python -m pytest grocery_organizer/tests # Python (chapter 3)
CI=true npm test                                     # JavaScript (chapter 5)

# Mobile (after web changes)
npm run ios:build && npm run ios:open
npm run android:build && npm run android:open
```

12.6 Conventions to know before contributing

- **Route logic lives in blueprints**, never in `api_server.py` / `api/index.py`.
- **Kroger is the only store chain**, used directly via `KrogerAPI` — the multi-chain abstraction was deliberately removed; don't reintroduce a chain parameter without a real second chain.
- **The backend speaks markdown** (`## Section\n- item`); the frontend owns presentation.
- **Colors only via `--af-*` CSS variables** (green palette + amber accent, dark mode via `prefers-color-scheme`); **Font Awesome icons, never emojis**; font set once on the shell. Chapter 6 and `docs/design-rules.html` are the law here.
- **Secrets only via environment variables** — nothing sensitive is ever committed; every credential in this guide flows through `.env` locally and the Vercel dashboard in production.
- **Guest mode must keep working** — every cloud feature degrades gracefully when its env vars are absent.

13 Security & App Hardening

A hobby app with real users still has real things to protect: your API credentials, your API quota, and your users' data. This chapter walks through the hardening that's actually in the codebase — each piece exists because of a concrete way the app could be abused, so the threat is explained alongside the defense.

13.1 The threat model

For an app this size, the realistic risks aren't nation-states — they're:

1. **Credential leakage** — the Kroger client ID/secret or AWS keys ending up somewhere public. The repo *is* public, so anything committed is published.
2. **Quota abuse** — the grocery endpoints are unauthenticated by design (guest mode is a feature), and every request spends Kroger API quota: 10,000 Products calls and 1,600 Locations calls per day. One hostile script can lock every real user out until midnight.
3. **Data exposure** — synced lists live in DynamoDB behind share codes and Cognito accounts; bugs in authorization logic leak other people's lists.
4. **Information leakage** — error messages and debug tooling revealing internals that make the other three easier.

Everything below maps back to one of these.

13.2 Credentials live only in the environment

Both halves of the Kroger OAuth pair come from env vars — the client ID is *not* hardcoded, because Kroger's API terms explicitly class client IDs as developer credentials that "may not be embedded in open source projects":

```
# kroger.py - get_auth_token()
client_id = os.getenv('KROGER_CLIENT_ID')
client_secret = os.getenv('KROGER_CLIENT_SECRET')
if not client_id or not client_secret:
    raise ValueError("KROGER_CLIENT_ID and KROGER_CLIENT_SECRET environment variables are required")
```

Locally they sit in `.env` (git-ignored); in production they're typed into the Vercel dashboard (chapter 7). The same pattern covers the AWS credentials (chapter 10).

Two hard-won lessons about git and secrets:

- **Deleting a committed secret doesn't unpublish it.** A file removed in a later commit is still one `git show` away in history. If a secret ever lands in a public repo, treat it as burned: rotate it at the provider, *then* worry about scrubbing history.
- **Rotation beats scrubbing.** Rewriting public history (`git filter-repo` + force push) breaks clones and doesn't recall caches or forks. A deactivated credential is worthless no matter who has it.

13.3 Bounding what one request can cost

`/api/process-grocery-list` fans out 1–2 Kroger calls *per list item*, in parallel. Without limits, a single upload with ten thousand lines drains the day's quota. So the route bounds the work before doing any of it:

```
# grocery_routes.py
MAX_ITEMS_PER_REQUEST = 100      # a big real-world list is ~50–60 items
MAX_UPLOAD_BYTES = 64 * 1024

raw = file.read()
if len(raw) > MAX_UPLOAD_BYTES:
    return jsonify({'error': 'List file is too large'}), 413
item_count = len(InputParser.parse_text(text))
if item_count > MAX_ITEMS_PER_REQUEST:
    return jsonify({'error': f'Lists are limited to {MAX_ITEMS_PER_REQUEST} items (got {item_cou
```

The principle: **validate cost, not just correctness**. The upload was always "valid" — the problem was what it would cost to serve. The same idea appears in `lists_backend.py` (`MAX_LIST_BYTES` keeps synced lists under DynamoDB's 400 KB item limit) and as an app-wide backstop in `api_server.py` (`MAX_CONTENT_LENGTH`). Note the ordering, too: the checks run *before* the Kroger calls, so rejected requests cost nothing.

The caps live in the blueprint, not in `GroceryListProcessor` — the CLI (you, locally) stays uncapped while both servers get the protection.

13.4 Rate limiting: `rate_limit.py`

Per-request caps don't stop someone looping requests, so every Kroger-backed endpoint sits behind a per-IP sliding window (30 requests/minute by default) via the `@rate_limited` decorator. The module is small enough to read in one sitting, and its details teach general lessons:

Trust the proxy, not the client. The client's address comes from the `X-Forwarded-For` header, but clients can *send* that header themselves. Proxies append the real address to the end, so only the last entry is trustworthy:

```
# rate_limit.py - _client_ip()
forwarded = request.headers.get('X-Forwarded-For', '')
last = forwarded.rsplit(',', 1)[-1].strip()
return last or request.remote_addr or 'unknown'
```

Taking the *first* entry — the intuitive choice — would let an attacker rotate through unlimited fake addresses and never hit the limit.

Defenses need their own defenses. The limiter tracks a deque of timestamps per IP. An attacker rotating addresses would grow that table forever — a memory leak *caused by* the rate limiter. Past a threshold, expired buckets get swept.

Scarcer quotas get smaller budgets. The Locations API allows 1,600 calls/day — 16% of the Products quota — and picking a store happens once per trip, not once per item. So store search declares its own budget:

```
@rate_limited(max_requests=10, bucket='locations')
def find_stores():
```

The `bucket` keeps its hits in a separate per-IP window, so a burst of product lookups can't lock someone out of store search, and vice versa. Match the limit to how the endpoint is *actually used*, not to one global number.

Know your limits' limits. The state is in-memory and per-process: full protection on a single-process server, but on Vercel each serverless instance keeps its own window, so the effective limit is (30 × instances). That's documented in the module docstring rather than hidden — a partial defense you understand beats a perfect one you assumed.

13.5 Shrinking the attack surface

Debug routes are gated. `api_server.py` has `/api/debug-kroger` and `/debug` — unauthenticated routes that make real Kroger calls and dump raw responses. Handy locally; a quota-burning gift on the public internet if this file is ever deployed — "it's in the local server file" is *not* the same as "it's local-only". The routes are only registered when `FLASK_ENV=development`, and `test_api_server.py` pins that behavior so it can't silently regress.

CORS is an allowlist everywhere. Both entry points share one origin list (`cors_config.py`): the CRA dev server, the production domain, and the Capacitor origins the mobile wrappers present. Before this, the Vercel entry point used `CORS(app)` — any website on the internet could call the API (and spend the quota) from its visitors' browsers. Same shared-module trick as the blueprints: one definition, so the two servers can't drift.

13.6 Errors that don't overshare

Exception text can contain file paths, library internals, even request URLs. Clients get a generic message; the log gets everything:

```
# grocery_routes.py
def _server_error(context, exc):
    # Exception details stay in the server log; clients get a generic message
    print(f"Error {context}: {exc}")
    traceback.print_exc()
    return jsonify({'error': f'Server error while {context}'}), 500
```

Related: malformed input should be a 400, not a 500. The JSON routes parse with `get_json(silent=True)` and validate field types (`_json_body()` / `_clean_item()`), so garbage bodies get clean rejections instead of tracebacks.

13.7 Multi-user boundaries

The list-sync endpoints (chapter 10) enforce three rules worth naming:

- **Membership is checked on every operation** — read, write, delete, share. There's no "the client wouldn't send that" — the client is whatever `curl` says it is.
- **Server-owned fields are stripped from client input.** `put_list` discards any `members` / `shareCode` the client sends; otherwise a member could promote themselves or forge a share code.
- **Share codes resist guessing.** Six characters from a 31-character alphabet (~890M combinations), generated with `random.SystemRandom` — the cryptographically seeded generator, not the predictable default `random` — and `/api/lists/join` sits behind the rate limiter, applied *before* auth so guessing attempts can't even spend Cognito lookups. At 30 attempts/minute, the code space is out of reach.

Passwords never touch this codebase at all — Cognito owns sign-up, verification, and reset (chapter 10). The best password-handling code is none.

13.8 Privacy and API compliance

Security also means keeping promises:

- **Privacy policy** — `public/privacy.html`, linked from the account sheet. Kroger's API terms require one for any client collecting user info, and it must stay accurate: if the app starts collecting something new, the page changes in the same PR.
- **Kroger's terms, distilled**: don't embed credentials in the repo, don't store API content permanently, respect the rate limits, display required attributions, don't imply Kroger endorses the app. Product images are hotlinked from Kroger's CDN rather than stored partly for this reason.

TODOs to get this working

- ☐ **Set** `KROGER_CLIENT_ID` alongside `KROGER_CLIENT_SECRET` in the Vercel dashboard — lookups fail without both
- ☐ **Verify** `FLASK_ENV` is not `development` on any deployed server, so the debug routes and Flask debug mode stay off
- ☐ **Deactivate retired Kroger clients** in the developer portal — any credential that ever appeared in a public commit is burned
- ☐ **Read** `public/privacy.html` and confirm it matches what the app actually collects
- ☐ **Occasionally run** `npm audit` / `pip list --outdated` — CRA's dev-dependency noise makes this low-signal, but real issues in runtime deps do surface this way